



GPGPU Workshop

Ziua IV

Proiect. Sugestii de rezolvare.
Simulări fizice. 😊

Mihnea-Petruț-Ilie Mitrache

04.08.2026



Organizare Workshop

1 Introducere

- Metoda GPGPU
- Ecosistemul CUDA
- GPU vs CPU
- Arhitectura paralelă
- Setup

2 Indexare thread-uri

- Arhitectură paralelă în detaliu
- Aritmetică thread
- Sample-uri de cod
- Primele kernel-uri

3 Prelucrare imagini

- Exemplu complex
- Studiu problemă în spiritul GPGPU
- Utilitare importante

4 Proiect

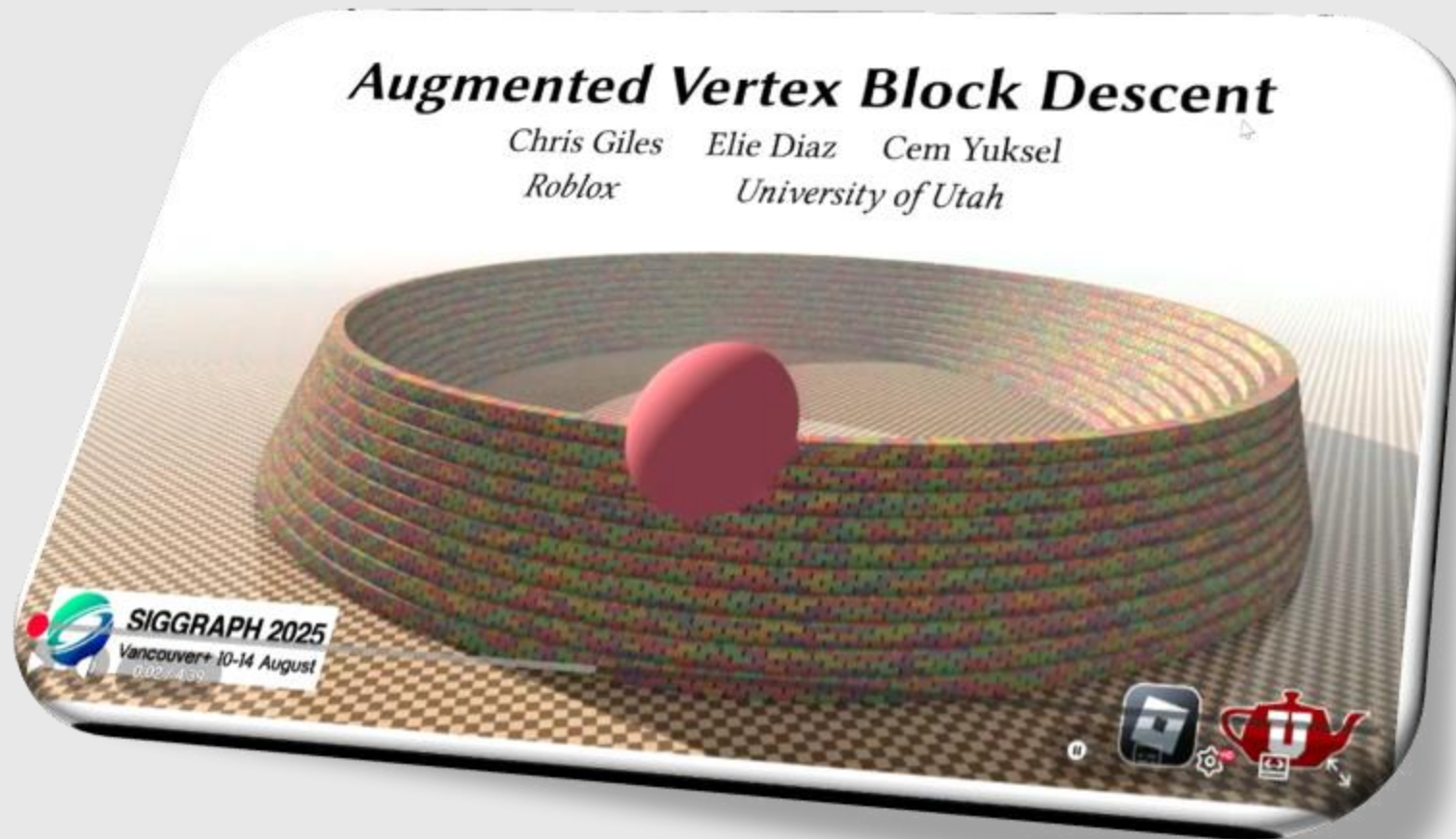
- Explicare temă
- Sugestii legate de rezolvare
- Listă de materiale pentru înțelegere mai bună

ASTĂZI

CPU vs GPU



Simulări fizice



Simulări fizice

- <https://www.youtube.com/watch?v=7NF3CdXkm68> (~ 7:50 min)

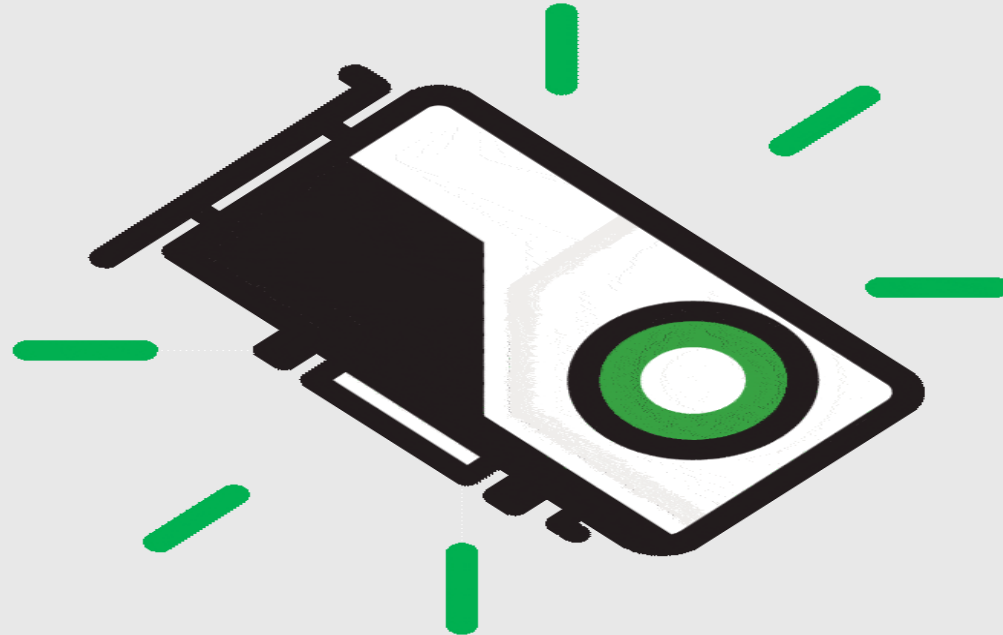
Simulări fizice

- <https://www.youtube.com/watch?v=2HCgKfKy3W8> (~ 5:30 min)

Simulări fizice

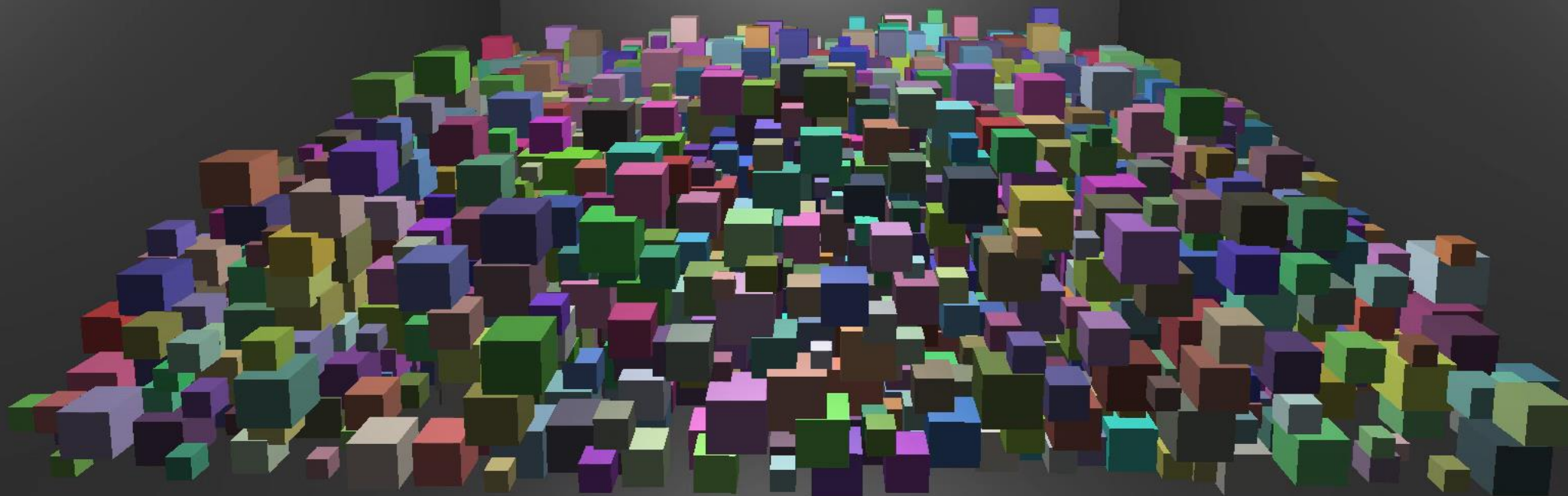
- <https://www.youtube.com/watch?v=bwJgifqvvd5M> (~ 4:40 min)

Demo Time 😊 - Proiectul vostru



=== Performance Stats ===
FPS: 166.7 (6.00 ms)
Objects: 5006
Collisions Detected: 0
Physics Time: 0.0001 ms
Controls: R=Reset Space=Pause G=Toggle GPU

Ce credeți că face GPU-ul
aici ? (2 perspective)



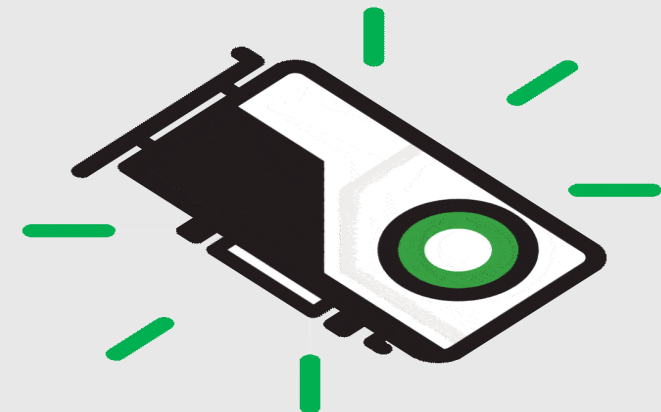
Cum vom lucra ?

- Framework vs Engine

Care e mai bun ? 😊

Avantaje, Dezavantaje

- UPB GFX Framework

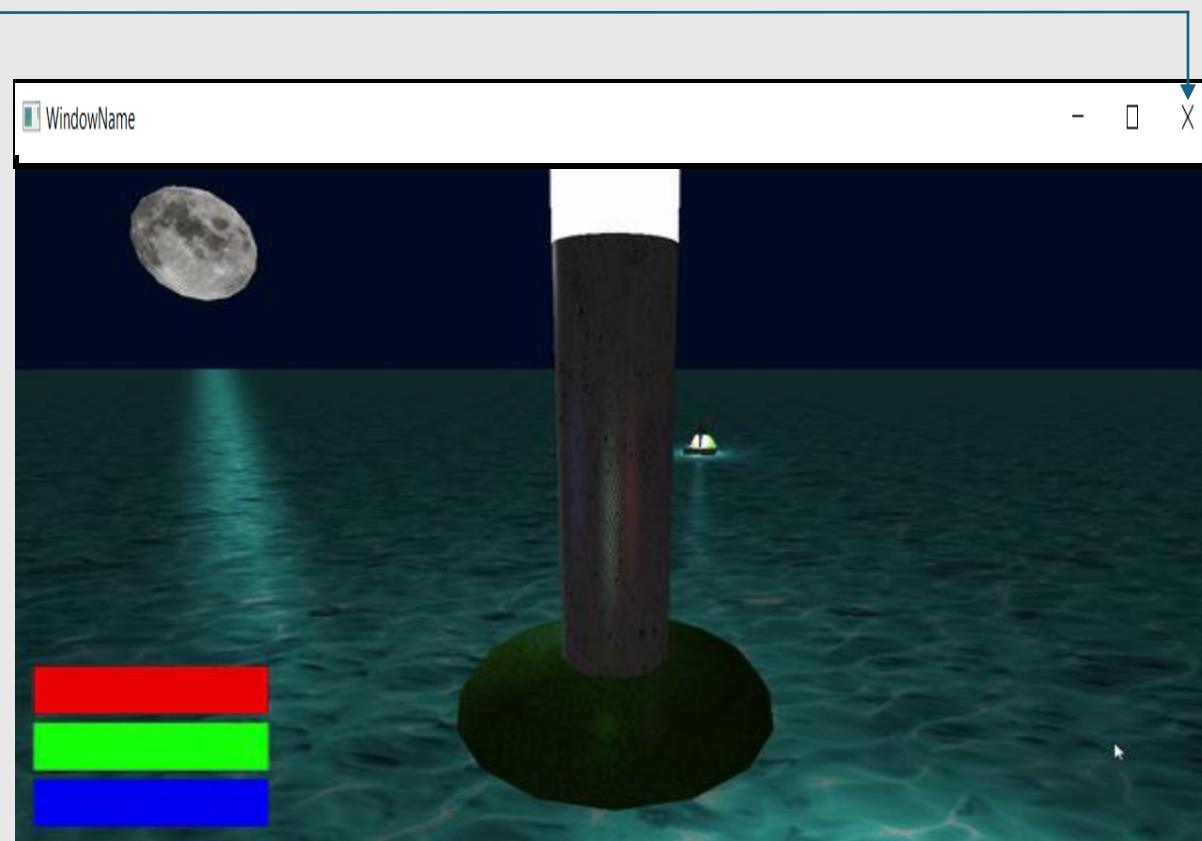
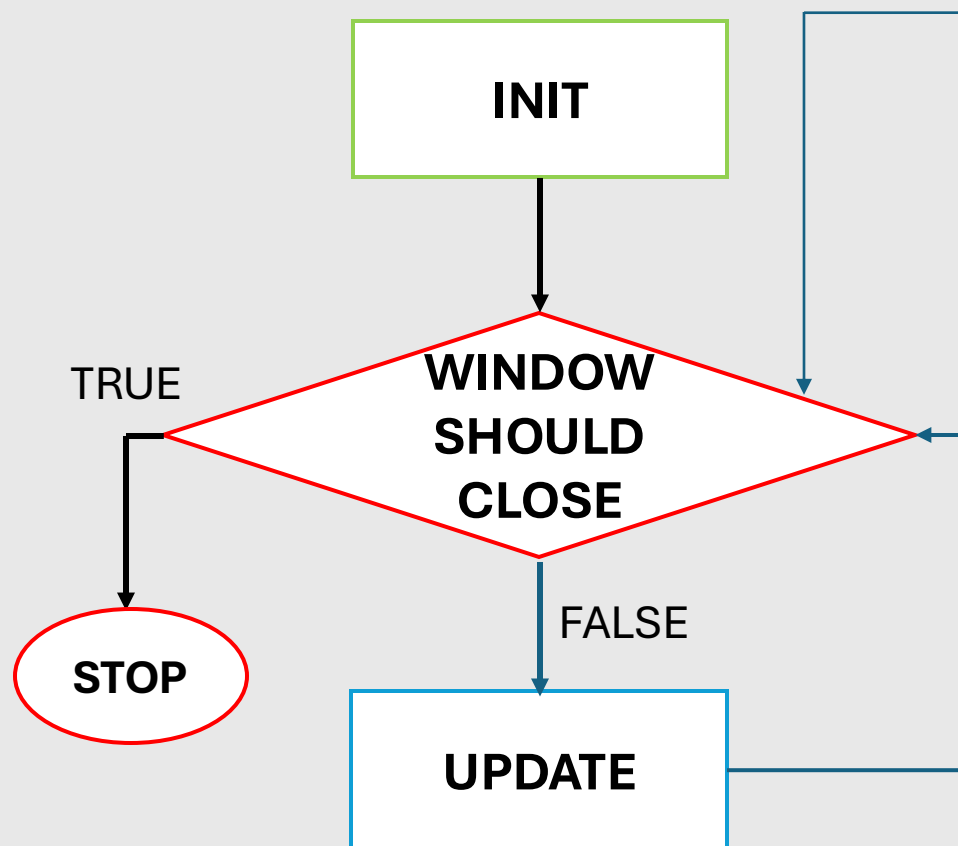


Framework

- Bazat pe OpenGL
- API grafic care permite programarea plăcii video
- Diferit față de conceptul de „bibliotecă”
- **Specificație** prin care se accesează codul deja existent din GPU (driver)

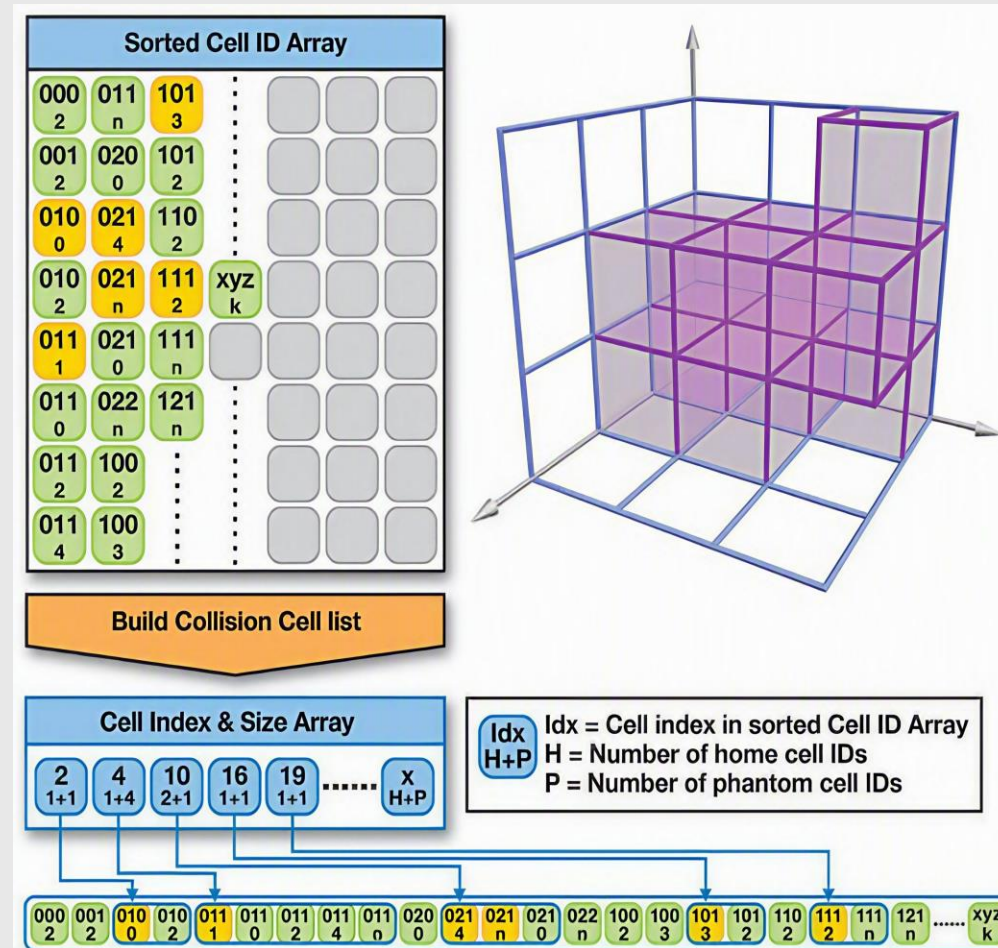


Maniera de lucru – Aplicație continuă



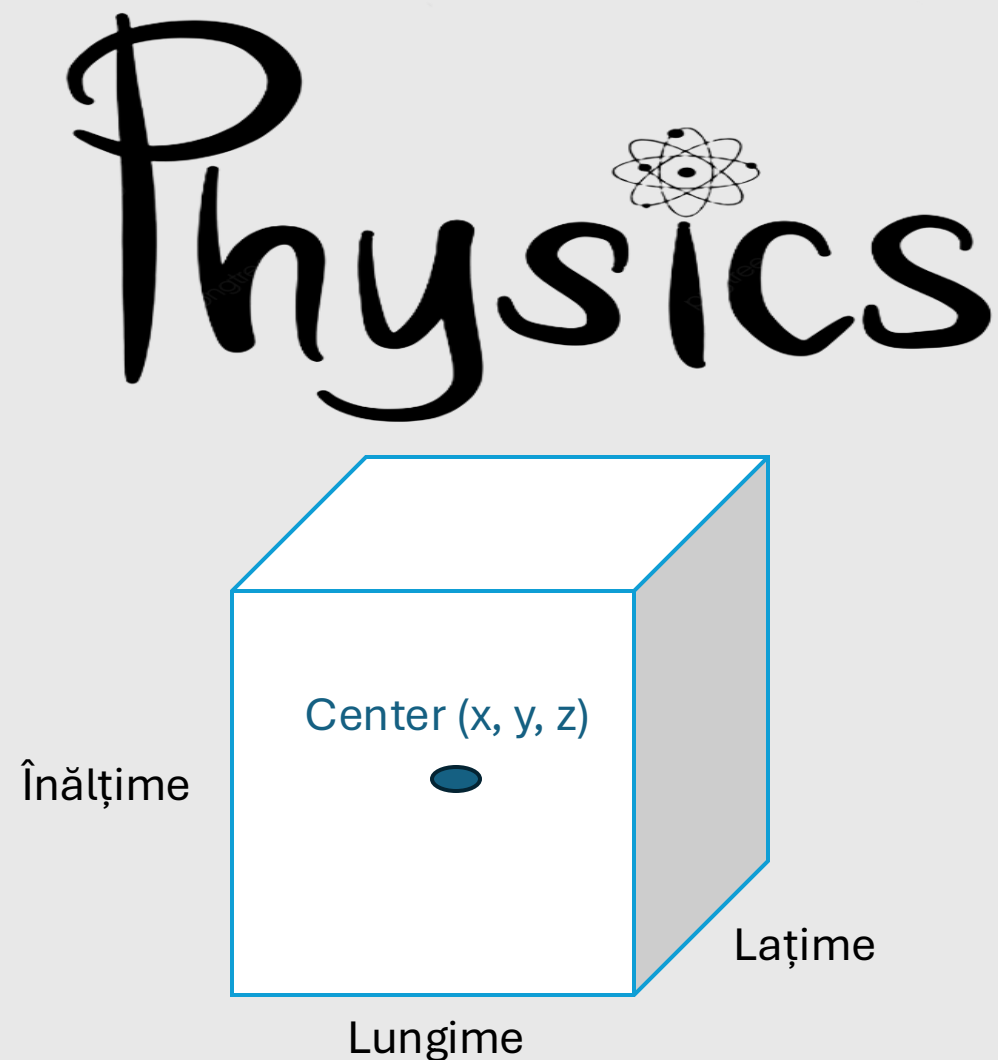
Proiectul vostru - Descriere

- Simulare coliziuni pentru mai multe cuburi în mediu comun cu accelerare GPGPU (CUDA) – Demo Slide 9
- Coliziune AABB (Axis-Aligned Bounding Box) sau OOB (Object-Oriented Bounding Box)
- CPU serial mai întâi
- GPGPU (CUDA) după implementarea serială



Model de date - Cub

- Obiect fizic (Proprietăți asociate)
 - Masă
 - Coeficient de frecare
 - Coeficient de elasticitate
- Proprietăți geometrice
 - Volum încadrator
 - Coordonate centru
 - Lungime laturi



Simularea

- N cutii. Câte posibile coliziuni avem ?

Fiecare cu fiecare $\frac{Nx(N-1)}{2} + N * 6$

- Fizică independentă de frame rate (Nu rulează sincron cu randarea)
- Doar detectarea coliziunilor se accelerează pe GPU!



Detectare coliziuni (Broad Phase)

- Verificarea de tipul “fiecare cu fiecare” este costisitoare. $O(N^2)$
- 500 de obiecte -> 127,750 verificări în fiecare cadru
- Ce credeți că face Broad Phase ?

Elimina coliziuni la scară largă. Cu ce seamănă ?

BVH (Ray Tracing)

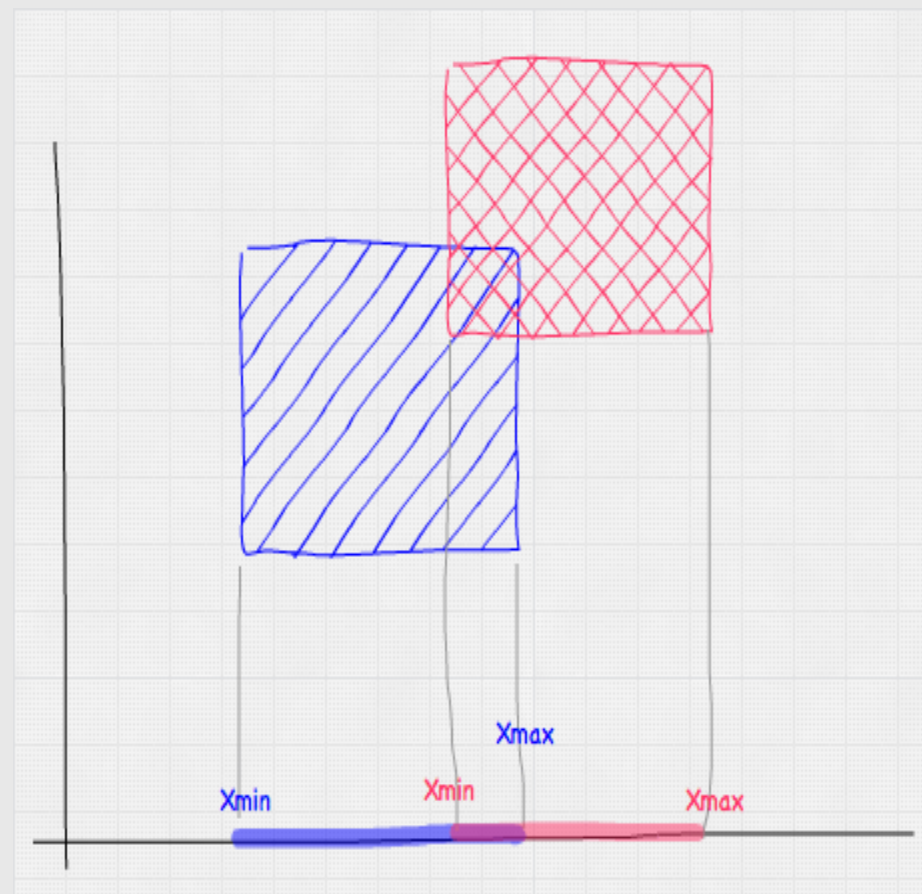
- Soluții potențiale (la alegerea voastră)
 - Sfere de încadrare
 - Grid spațial
 - Sortare
- Orice soluție ați alege – Nu eliminați coliziuni reale! (Fară Fals pozitiv)

Detectare coliziuni (Narrow Phase)

- Ce credeți că este ?

Fază de verificare în detaliu care stabilește coliziunile reale

- 10.000 de coliziuni potențiale
 - 7000 eliminate de Broad Phase
 - restul (3000) sunt tratate de Narrow Phase
- Coliziuni
 - Cub-cub
 - Cub-plan (perete). Foarte asemănător!



Răspuns coliziune

- Tot timpul pe CPU! De ce credeți că e mai bine așa ?

Detectarea coliziunii este inerent paralelă.

Răspunsul este exact opusul, vizează individual entitățile care se lovesc.
Actualizez poziții independente!

- Suprapunerea determină puterea coliziunii
 - Separă - Împinge cele două cutii una departe de cealaltă, de-a lungul axei de lovire, astfel încât să nu mai se suprapună.
 - Ricoșează - Aplică un impuls scalat de elasticitatea fiecărei cutii
 - Frecare - Aplică un impuls de frecare tangent la punctul de contact, încetinind mișcarea relativă de alunecare.
- Potețiale noi coliziuni imediat după. De ce credeți că apar ?

Rezolvarea perechii A-B poate reintroduce coliziunea în perechea B-C

Coliziuni – Teorie importanti

- https://developer.mozilla.org/en-US/docs/Games/Techniques/3D_collision_detection
- <https://www.toptal.com/game/video-game-physics-part-ii-collision-detection-for-solid-objects>
- <https://iopscience.iop.org/article/10.1088/1742-6596/1213/4/042079/pdf>
- https://www.gamasutra.com/view/feature/131790/simple_intersection_tests_for_games.php?page=5



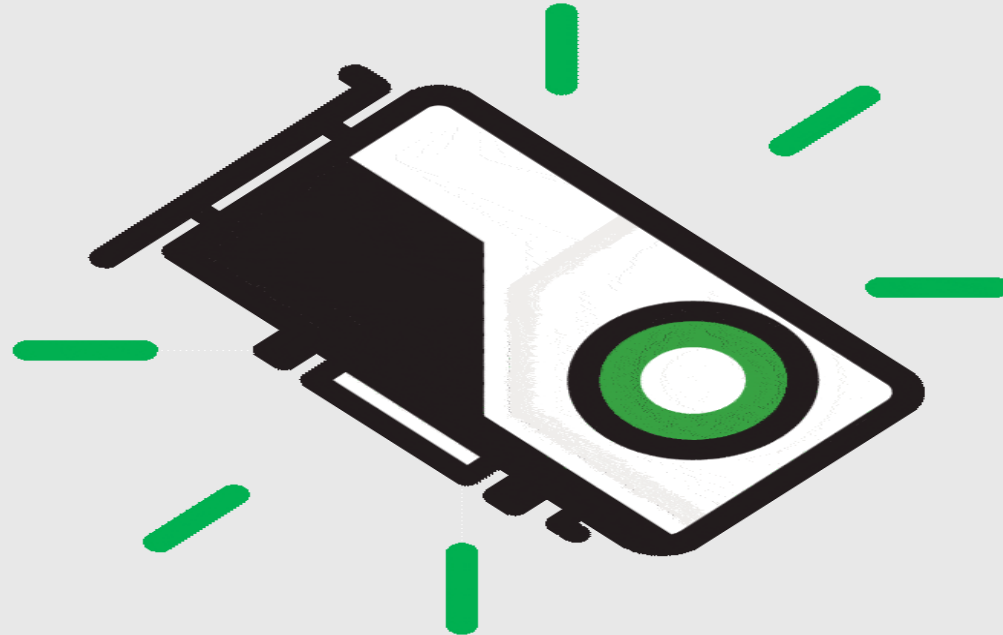
Unde lucrăm - Schelet ?

- Reminder UPB-GFX
- Varintă care include CUDA ca dependență
- [Repo](#)
- Obligatoriu CUDA!

```
# TODO: Set your CUDA architecture here. Check https://developer.nvidia.com/cuda-gpus for a list of
supported architectures.
if (NOT CMAKE_CUDA_ARCHITECTURES)
    set(CMAKE_CUDA_ARCHITECTURES 120)
endif()

# Set the name of the project
set(target_name GFXFramework)
project(${target_name} C CXX CUDA)
```

Demo Time 😊 - Schelet



Alte aplicații CUDA

- NVIDIA PhysX FleX
 - <https://www.youtube.com/watch?v=1o0Nuq71gl4>
 - https://www.youtube.com/watch?v=pfonMfP_Ks
 - <https://www.youtube.com/watch?v=Jl54WZtm0QE>
- NVIDIA IndeX
 - <https://www.youtube.com/watch?v=GAZP1NcdWMo>
- NVIDIA TensorRT 3:
 - <https://www.youtube.com/watch?v=8WS7VZ4UAZw>
- Weather simulations
 - <https://www.youtube.com/watch?v=tPx1MLMVRog>
- OpenCL applications
 - <https://www.youtube.com/watch?v=HR7SgCKu8Oc>
- Autonomous driving:
 - <https://www.youtube.com/watch?v=0rc4RqYLtEU>
- NVIDIA CUDA 10 Samples
 - <https://www.youtube.com/watch?v=RgLueg1o4Kc&t=232s>